API Guide
User Manual.

Version 1.0.0

Edilson Filho

Correspondent: edilsonfilho@lesc.ufc.br

Contents

1 Introduction **2**
1.1 What is TV White Space? . . . 2
1.2 The TVWS-DB API . . . 2
1.3 RFC 7545 Compliance . . . 2
1.4 Base Endpoint . . . 3

2 Usage Flow **3**
2.1 Overview . . . 3
2.2 JSON-RPC 2.0 Message Structure . . . 3
2.3 Step 1 — INIT_REQ . . . 4
2.4 Step 2 — REGISTRATION_REQ . . . 4
2.5 Step 3 — AVAIL_SPECTRUM_REQ . . . 4

3 Message Reference **4**
3.1 INIT_REQ . . . 4
3.2 REGISTRATION_REQ . . . 5
3.3 AVAIL_SPECTRUM_REQ . . . 9

4 Error Codes **15**
4.1 Protocol Errors — $-1xx$ . . . 16
4.2 Parameter Errors — $-2xx$ . . . 16
4.3 Authentication and Access Errors — $-3xx$ . . . 17
4.4 Server Errors — $-5xx$ . . . 18

References **19**

1 Introduction

1.1 What is TV White Space?

TV White Space (TVWS) refers to unused portions of radio-frequency spectrum in the television broadcasting bands (VHF/UHF, roughly 54–698 MHz). These spectrum gaps arise geographically and temporally from the digital television transition and the uneven distribution of broadcast transmitters across a territory.

TVWS technology enables *secondary* devices—such as broadband internet access equipment—to operate on idle TV channels without interfering with *primary* users (licensed television broadcasters and wireless microphones). Secondary access is coordinated through a geolocation database, which tracks which TV channels are available at any given location and time.

1.2 The TVWS-DB API

The TVWS-DB API is a server-side implementation of the Protocol to Access White-Space (PAWS) Databases, as specified in **RFC 7545** Mancuso et al., 2015. It exposes a single HTTP endpoint through which TVWS devices query spectrum availability for a given geographic location.

The API performs the following functions:

- Validates device identity (manufacturer, model, serial number, ruleset).
- Verifies the geographic coordinates of the requesting device.
- Enforces antenna parameter constraints (height, direction, radiation pattern).
- Returns the list of available TV channels and their maximum permitted transmission power for the queried location.

1.3 RFC 7545 Compliance

This API fully implements the PAWS protocol defined in RFC 7545 Mancuso et al., 2015. The PAWS framework specifies:

- A **JSON-RPC 2.0** message transport model over HTTP.
- Three mandatory request/response message types: `INIT_REQ`, `REGISTRATION_REQ`, and `AVAIL_SPECTRUM_REQ`.
- A standardized error reporting model with well-defined numeric error codes.
- Ruleset-based spectrum availability rules (e.g., `FccTvBandWhiteSpace-2010`).

In addition to the standard PAWS error codes, this implementation defines a set of extension codes for authentication and session management (see Section 4).

1.4 Base Endpoint

All PAWS requests are sent as HTTP **POST** requests to:

```
POST /paws
```

The request body must be a valid JSON-RPC 2.0 object. The `Content-Type` header must be `application/json`.

Health Check

A lightweight health check endpoint is also available:

```
GET /paws/test
```

Returns HTTP 200 with body `"up"` when the server is running.

2 Usage Flow

2.1 Overview

The PAWS protocol defines a mandatory three-step sequence. Every TVWS device must complete these steps **in order** before obtaining spectrum availability data:

1. **INIT_REQ** — Device initialization. The database acknowledges the device and returns ruleset information for the device's location.
2. **REGISTRATION_REQ** — Device registration. The device provides owner and operator contact information (vCard format).
3. **AVAIL_SPECTRUM_REQ** — Spectrum availability query. The device sends its precise location and antenna parameters; the database returns the available TV channels with permitted power levels.

2.2 JSON-RPC 2.0 Message Structure

All PAWS messages use the JSON-RPC 2.0 envelope. Every request must include the four top-level fields shown below.

Field	Type	Description
`jsonrpc`	string	Protocol version. Must be `"2.0"`.
`method`	string	PAWS method name (varies per message type).
`params`	object	Message-specific parameters.
`id`	string	Client-assigned request identifier (e.g., `"2026.1a"`).

A successful response contains a `result` field. An error response contains an `error` field with the sub-fields `code`, `message`, and `data.parameters`.

2.3 Step 1 — INIT_REQ

Method: `spectrum.paws.init`

The initialization request introduces the device to the database. The device provides its identity (`deviceDesc`) and current location. On success, the database returns an `INIT_RESP` containing the applicable ruleset information.

2.4 Step 2 — REGISTRATION_REQ

Method: `spectrum.paws.register`

The registration request extends the initialization message with owner and operator contact information in jCard (vCard 4.0) format. This step must be completed once before any spectrum query is accepted.

2.5 Step 3 — AVAIL_SPECTRUM_REQ

Method: `spectrum.paws.getSpectrum`

This is the primary query. The device provides its current location, antenna parameters, and optionally master device information. The database returns the list of available TV channels with their maximum permitted transmission power for the queried location.

This request can be repeated as often as needed (subject to polling limits defined in the `INIT_RESP` ruleset). The device must re-send registration data (`deviceOwner`) with each request.

3 Message Reference

This section documents the complete field reference for each PAWS message type. Fields are expressed in dot notation (e.g., `params.location.point.center.latitude`).

The **Req.** column uses:

- **Yes** — field must be present and valid.
- **No** — field is optional.
- **Cond.** — required only when a related field is present (see Constraints column).

3.1 INIT_REQ

Method: `spectrum.paws.init`

Listing 1: INIT_REQ — complete example

```
{
  "jsonrpc": "2.0",
  "method": "spectrum.paws.init",
  "params": {
    "type": "INIT_REQ",
    "version": "1.0",
```

```
    "deviceDesc": {
      "serialNumber": "4fbf-bae3-c8c5706745e3",
      "fccId": "94d8610a-9eea",
      "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",
      "modelId": "xpto-123-123-12",
      "rulesetIds": ["FccTvBandWhiteSpace-2010"]
    },
    "location": {
      "point": {
        "center": {
          "latitude": -3.752542,
          "longitude": -38.556012
        }
      }
    }
  },
  "id": "2026.1a"
}
```

Field	Type	Req.	Constraints
`jsonrpc`	string	Yes	Must be `"2.0"`
`method`	string	Yes	`"spectrum.paws.init"`
`id`	string	Yes	Request identifier
`params.type`	string	Yes	`"INIT_REQ"`
`params.version`	string	Yes	`"1.0"`
`params.deviceDesc.serialNumber`	string	Yes	Device serial number
`params.deviceDesc.fccId`	string	Yes	FCC identifier
`params.deviceDesc.manufacturerId`	string	Yes	Manufacturer identifier
`params.deviceDesc.modelId`	string	Yes	Model identifier
`params.deviceDesc.rulesetIds`	string[]	Yes	e.g. `["FccTvBandWhiteSpace-2010"]`
`params.location.point.center.latitude`	number	Yes	Range: $[-90, 90]$
`params.location.point.center.longitude`	number	Yes	Range: $[-180, 180]$

3.2 REGISTRATION_REQ

Method: `spectrum.paws.register`

`REGISTRATION_REQ` includes all fields from `INIT_REQ` (Section 3.1), with `method` and `params.type` changed as shown, plus the additional `deviceOwner` object.

Listing 2: REGISTRATION_REQ — example (deviceOwner section)

```
{
   "jsonrpc":"2.0",
```

```
  "method":"spectrum.paws.register",
  "params":{
    "type":"REGISTRATION_REQ",
    "version":"1.0",
    "deviceDesc":{
      "serialNumber":"4fbf-bae3-c8c5706745e3",
      "fccId":"94d8610a-9eea",
      "manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3",
      "modelId":"xpto-123-123-123",
      "rulesetIds":[
        "FccTvBandWhiteSpace-2010"
      ]
    },
    "location":{
      "point":{
        "center":{
          "latitude":-4.978806,
          "longitude":-39.056607
        }
      }
    },
    "deviceOwner":{
      "owner":{
        "vcard":[
          [
            "version",
            {

            },
            "text",
            "4.0"
          ],
          [
            "fn",
            {

            },
            "text",
            "Fractal Networks"
          ],
          [
            "org",
            {

            },
            "text",
            "Fractal Networks"
          ],
```

```
            [
               "adr",
               {

               },
               "text",
               [
                  "",
                  "",
                  "123 Innovation Ave",
                  "San Francisco",
                  "CA",
                  "94107",
                  "USA"
               ]
            ],
            [
               "email",
               {

               },
               "text",
               "contact@fractalfn.com"
            ],
            [
               "tel",
               {

               },
               "uri",
               "tel:+1-415-555-0100"
            ]
         ]
      },
      "operator":{
         "vcard":[
            [
               "version",
               {

               },
               "text",
               "4.0"
            ],
            [
               "fn",
               {

```

```
                },
                "text",
                "Jane Doe"
            ],
            [
                "org",
                {

                },
                "text",
                "Fractal Networks"
            ],
            [
                "adr",
                {

                },
                "text",
                [
                    "",
                    "",
                    "123 Innovation Ave",
                    "San Francisco",
                    "CA",
                    "94107",
                    "USA"
                ]
            ],
            [
                "email",
                {

                },
                "text",
                "jane.doe@fractalfn.com"
            ],
            [
                "tel",
                {

                },
                "uri",
                "tel:+1-415-555-0101"
            ]
        ]
    }
  }
},
```

```
  "id":"10"
}
```

Field	Type	Req.	Constraints
`method`	string	Yes	`"spectrum.paws.register"`
`params.type`	string	Yes	`"REGISTRATION_REQ"`
All INIT_REQ fields also apply (Section 3.1).			
`params.deviceOwner.owner.vcard`	array	Yes	jCard 4.0 array (see note below)
`params.deviceOwner.operator.vcard`	array	Yes	jCard 4.0 array (see note below)

vCard format. The `vcard` field follows the jCard specification (RFC 7095). Each entry is a four-element array: `[property, parameters, type, value]`. Supported properties: `version` (must be `"4.0"`), `fn` (full name), `org` (organization), `adr` (address), `email`, and `tel`.

3.3 AVAIL_SPECTRUM_REQ

Method: `spectrum.paws.getSpectrum`

`AVAIL_SPECTRUM_REQ` includes all fields from `REGISTRATION_REQ` (Section 3.2), with `method` and `params.type` changed as shown, plus the mandatory `antenna` object and optional master device fields.

Listing 3: AVAIL_SPECTRUM_REQ — antenna and master device fields

```
{
  "jsonrpc":"2.0",
  "method":"spectrum.paws.getSpectrum",
  "params":{
    "type":"AVAIL_SPECTRUM_REQ",
    "version":"1.0",
    "deviceDesc":{
      "serialNumber":"4fbf-bae3-c8c5706745e3",
      "fccId":"94d8610a-9eea",
      "manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3",
      "modelId":"xpto-123-123-123",
      "rulesetIds":[
        "FccTvBandWhiteSpace-2010"
      ]
    },
    "masterDeviceDesc":{
      "serialNumber":"4fbf-bae3-c8c5706745e3",
      "fccId":"94d8610a-9eea",
      "manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3",
```

```
      "modelId":"xpto-123-123-123",
      "rulesetIds":[
         "FccTvBandWhiteSpace-2010"
      ]
   },
   "location":{
      "point":{
         "center":{
            "latitude":-3.7653855,
            "longitude":-38.5261224
         }
      }
   },
   "deviceOwner":{
      "owner":{
         "vcard":[
            [
               "version",
               {

               },
               "text",
               "4.0"
            ],
            [
               "fn",
               {

               },
               "text",
               "Fractal Networks"
            ],
            [
               "org",
               {

               },
               "text",
               "Fractal Networks"
            ],
            [
               "adr",
               {

               },
               "text",
               [
                  "",
```

```
                "",
                "123 Innovation Ave",
                "San Francisco",
                "CA",
                "94107",
                "USA"
            ]
        ],
        [
            "email",
            {

            },
            "text",
            "contact@fractalfn.com"
        ],
        [
            "tel",
            {

            },
            "uri",
            "tel:+1-415-555-0100"
        ]
    ]
},
"operator":{
    "vcard":[
        [
            "version",
            {

            },
            "text",
            "4.0"
        ],
        [
            "fn",
            {

            },
            "text",
            "Jane Doe"
        ],
        [
            "org",
            {

```

```
                },
                "text",
                "Fractal Networks"
            ],
            [
                "adr",
                {

                },
                "text",
                [
                    "",
                    "",
                    "123 Innovation Ave",
                    "San Francisco",
                    "CA",
                    "94107",
                    "USA"
                ]
            ],
            [
                "email",
                {

                },
                "text",
                "jane.doe@fractalfn.com"
            ],
            [
                "tel",
                {

                },
                "uri",
                "tel:+1-415-555-0101"
            ]
        ]
    }
},
"antenna":{
    "height":70,
    "heightType":"AGL",
    "antennaDirection":{
        "zenith":10.1,
        "azimuth":12.1
    },
    "antennaRadiationPattern":[
        [
```

```
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
```

```
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0,
                0.0
             ]
          ],
          "antennaPolarization":[
             "h"
          ]
       },
       "masterDeviceLocation":{
          "point":{
             "center":{
                "latitude":-3.7653855,
                "longitude":-38.5261224
             }
          }
       }
    },
    "id":"10"
}
```

Field	Type	Req.	Constraints
`method`	string	Yes	`"spectrum.paws.getSpectrum"`
`params.type`	string	Yes	`"AVAIL_SPECTRUM_REQ"`

(continued)

Field	Type	Req.	Constraints
All REGISTRATION_REQ fields also apply (Section 3.2).			
`params.antenna.height`	number	Yes	AGL: $[-200,\ 2000]$ m AMSL: $[-3500,\ 8848]$ m
`antenna.heightType`	string	Yes	`"AGL"` or `"AMSL"`
`antenna.antennaDirection.zenith`	number	Cond.	$[-90,\ 90]$; required when `antennaRadiationPattern` is present
`antenna.antennaDirection.azimuth`	number	Cond.	$[0,\ 360]$; required when `antennaRadiationPattern` is present
`antenna.antennaPolarization`	string[]	Cond.	`["h"]`, `["v"]`, or `["h","v"]`; required when `antennaRadiationPattern` is present
`antenna.antennaRadiationPattern`	number[][]	No	1 sub-array (single pol.) or 2 sub-arrays (dual pol.); each must contain exactly **72** numeric values
`params.masterDeviceDesc`	object	No	Same structure as `params.deviceDesc`
`params.masterDeviceLocation`	object	No	Same structure as `params.location`

Height types.

- **AGL** (Above Ground Level): height measured from the ground surface directly below the antenna. Valid range: -200 to 2000 metres.
- **AMSL** (Above Mean Sea Level): height measured from mean sea level. Valid range: -3500 to 8848 metres (Dead Sea floor to Mt. Everest peak).

Antenna radiation pattern. When `antennaRadiationPattern` is present, both `antennaDirection` and `antennaPolarization` become mandatory. The number of sub-arrays must match the polarization count: **1** sub-array for single polarization (`["h"]` or `["v"]`), **2** sub-arrays for dual polarization (`["h","v"]`). Each sub-array must contain exactly 72 numeric values representing the antenna gain (in dBi) at 5° intervals from 0° to 355°. Mismatches between polarization count and sub-array count return error `-201`.

4 Error Codes

When a request fails validation or cannot be processed, the API returns a JSON-RPC 2.0 error response. The `error` object structure is:

Listing 4: Error response structure

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -202,
    "message": "A parameter value is invalid in some way.",
    "data": {
      "parameters": ["params.antenna.height"]
    }
  },
  "id": "2026.1a"
}
```

The `data.parameters` array lists the specific fields that triggered the error. A successful response contains a `result` field instead of `error`.

Error codes are grouped by numeric range. Codes in the $-1xx$ and $-2xx$ ranges are defined by RFC 7545 Mancuso et al., 2015. Codes in the $-3xx$ and $-5xx$ ranges are API extensions.

4.1 Protocol Errors — $-1xx$

Returned when the request cannot be understood or the device/ruleset is not supported.

Code	Name	Origin	Description
-101	UNSUPPORTED	RFC 7545	The database does not support the specified protocol version (`params.version`).
-102	UNSUPPORTED	RFC 7545	The database does not support the device type or the specified ruleset (`params.deviceDesc.rulesetIds`).
-103	UNIMPLEMENTED	RFC 7545	The database does not implement the requested optional feature.
-104	OUTSIDE_COVERAGE	RFC 7545	The device's location is outside the database's coverage area.
-105	DATABASE_CHANGE	RFC 7545	The database URI has changed. The device must update its list of available databases.
-106	SESSION_ERROR	Extension	An error occurred in the session layer.
-107	DATABASE_UNAVAILABLE	Extension	The database backend is not available at this time.

4.2 Parameter Errors — $-2xx$

Returned when request parameters are missing or contain invalid values. These are the most common errors during integration.

Code	Name	Origin	Description
-201	MISSING	RFC 7545	A required parameter is absent from the request. The `data.parameters` array lists each missing field path.
-202	INVALID_VALUE	RFC 7545	A parameter is present but its value is invalid: wrong type, out of permitted range, or logically inconsistent with another field. The `data.parameters` array identifies the offending fields.

Common -202 triggers

Field	Condition that triggers -202
`params.type`	Value does not match the `method` field.
`params.location.point.center.latitude`	Not a number, or outside $[-90,\ 90]$.
`params.location.point.center.longitude`	Not a number, or outside $[-180,\ 180]$.
`params.antenna.height`	Not a number, or out of range for the given `heightType`.
`params.antenna.heightType`	Not `"AGL"` or `"AMSL"`.
`params.antenna.antennaDirection.azimuth`	Not a number, or outside $[0,\ 360]$.
`params.antenna.antennaDirection.zenith`	Not a number, or outside $[-90,\ 90]$.
`params.antenna.antennaDirection`	Absent when `antennaRadiationPattern` is present.
`params.antenna.antennaPolarization`	Absent when `antennaRadiationPattern` is present.
`params.antenna.antennaRadiationPattern`	Not an array, or a sub-array element is not a number.

4.3 Authentication and Access Errors — $-3xx$

Returned when authentication or session checks fail. These codes are API extensions to RFC 7545.

Code	Name	Origin	Description
-301	UNAUTHORIZED	Extension	The request is not authorized.
-302	NOT_REGISTERED	RFC 7545	The device must complete `REGISTRATION_REQ` before sending other requests.
-303	ACCESS_EXPIRED	Extension	The device's session or access token has expired. Re-authentication is required.

Code	Name	Origin	Description
−304	AUTHENTICATION_FAILED	Extension	The provided authentication credentials are invalid or missing.

4.4 Server Errors — $-5xx$

Code	Name	Origin	Description
−500	SERVER_ERROR	Extension	An unexpected internal error occurred on the server.

References

Mancuso, V., Probasco, S., & Patil, B. (2015, May). *Protocol to Access White-Space (PAWS) Databases* (RFC No. 7545). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC7545